Privacy-Preserving Employee Database Using Partial Homomorphic Encryption and Searchable Encryption

Francisco Sobral de Almeida - 75116
Advanced Applied Cryptography
FCT / NOVA

2026

Abstract

This report describes the design, implementation, and evaluation of a privacy-preserving employee database system. The system stores employee information in an encrypted MongoDB database while still supporting search, ordering, payroll computation, salary comparison, and bonus-related operations from the client side. The solution combines deterministic encryption, randomized authenticated encryption, order-preserving encryption, homomorphic encryption, digital signatures, and HMAC-based integrity protection.

Contents

1 Introduction

The goal of this project is to outsource an employee database to an untrusted server while keeping all sensitive information encrypted. The server must remain unable to learn employee plaintext values, yet still support a defined set of operations over encrypted data.

This project is based on the provided assignment requirements and implements the system in Java. The client is responsible for bootstrap, encryption, indexing, query execution, and integrity verification. The server only stores and returns encrypted records.

1.1 Project Objectives

- Store employee records encrypted in a remote database.
- Support encrypted search and retrieval by employee ID and full name.
- Support ordered queries over salary and age.
- Support department-based payroll computation using homomorphic encryption.
- Support salary comparison between employees.
- Support bonus preview and bonus application operations.
- Maintain authenticity and integrity using signatures and HMAC.

2 System Design Assumptions

2.1 Threat Model

The system assumes an honest-but-curious server. The database server stores and returns records correctly, but it is not trusted with plaintext data or cryptographic keys.

2.2 Trusted Components

The client application is trusted and performs all cryptographic operations. The client stores the keys required for encryption, decryption, indexing, ordering, and verification.

2.3 Untrusted Components

The MongoDB server is treated as untrusted. It may observe ciphertexts, encrypted indexes, record structure, and access patterns, but it must not learn plaintext employee data.

2.4 Design Goals

- Confidentiality of employee attributes through encryption of all sensitive fields before outsourcing them to the database.
- Support for useful operations over encrypted data, including search, ordering, comparison, and aggregation.
- Integrity and authenticity guarantees through HMAC-SHA256 and ECDSA-based verification mechanisms.
- Practical usability through a simple command-line client interface.
- Reproducible deployment through automated bootstrap, preparation, cleanup, and snapshot generation procedures.

3 Architecture

3.1 High-Level Overview

The architecture is composed of three main parts:

1. The **client application**, which loads configuration, generates or loads keys, bootstraps the encrypted dataset, and executes commands.
2. The **MongoDB database**, which stores only encrypted employee records.
3. The **client-side index and key store**, which contains state needed for encrypted search and ordering.

3.2 Data Flow

During bootstrap, the client reads the CSV dataset, encrypts the relevant attributes, signs and authenticates records, and writes them to MongoDB. At query time, the client generates the encrypted query tokens, sends them to MongoDB, receives matching records, and decrypts or verifies them locally when needed.

3.3 Main Modules

- `App`: interactive command interpreter.
- `EmployeeService`: business logic for bootstrap, queries, and updates.
- `MongoEmployeeRepository`: persistence layer for MongoDB.
- `EncryptionService`: encryption and cryptographic primitives.

- `RecordSigner` / `RecordVerifier`: authenticity and integrity protection.
- `RecordDecryptor`: decrypted presentation layer.
- `ClientEmployeeIndexStore`: stored searchable index.
- `KeyManager`: local key loading and generation.

4 Database Encryption Choices

4.1 Rationale

Different attributes require different cryptographic protections depending on the operation they must support. No single encryption scheme is enough for all required query types.

4.2 Encryption Strategy

Attribute / Use Case	**Technique**	**Purpose**
Employee ID / Full name lookup	Deterministic encryption / searchable indexing	Exact match queries
Salary and age ordering	Boldyreva Order-preserving encryption (OPE)	Sorting and comparison without plaintext
Sensitive fields not requiring search	Randomized AES-GCM	Confidential storage with semantic security
Salary totals / payroll	Paillier homomorphic encryption	Addition over encrypted values
Integrity and authenticity	HMAC-SHA256 + ECDSA signatures	Detect tampering and verify origin

4.3 Implementation Notes

The client uses a mixture of deterministic AES-CTR, AES-GCM, OPE state, and Paillier cryptosystem support. The stored records are kept encrypted in MongoDB, and the client maintains the necessary state to process queries and verify records.

5 Bootstrap Process and Index Generation

5.1 Bootstrap Workflow

The bootstrap process performs the following steps:

1. Load configuration from `application.properties`.
2. Load or create local keys.
3. Read employee rows from the CSV dataset.
4. Encrypt each field using the appropriate scheme.
5. Build searchable and order-preserving structures.
6. Sign and authenticate each encrypted record.
7. Store the resulting records in MongoDB.

5.2 Client-Side Index

The client stores auxiliary state in `client/index`. This includes cryptographic keys, OPE state, employee index information, and generated snapshot files.

5.3 Bootstrap Output

At the end of bootstrap, the database contains encrypted employee records only. The client can then execute the supported commands against the encrypted database.

6 Supported Operations

The application provides an interactive command interpreter that supports the assignment requirements and additional utility operations.

6.1 Query and Retrieval

- Search by employee ID.
- Search by employee full name.
- Search by department.
- Retrieve employees ordered by salary.
- Retrieve employees ordered by age.

- Find the employee with the highest salary.
- Find the oldest employee.

6.2 Aggregation and Comparison

- Compare salaries of two employees.
- Compute total payroll for a department.
- Obtain salary previews for one or more employees.

6.3 Bonus Updates

- Preview bonuses for eligible employees.
- Apply and persist a 25% bonus to a single employee.
- Apply and persist bonuses to all eligible employees.

6.4 Encrypted Snapshot

The `snapshot` command exports the encrypted database contents into a snapshot file for size comparison and experimentation.

7 Command-Line Interface

The client provides the following commands:

Command	Meaning
`bootstrap`	Load the CSV dataset, encrypt it, and store it in MongoDB.
`count`	Show the number of stored employee records.
`find-id <employeeId>`	Search an employee by ID.
`find-name <full name>`	Search an employee by full name.
`find-dept <departmentId>`	List employees in a department.
`salary-asc` / `salary-desc`	Order employees by salary.
`age-asc` / `age-desc`	Order employees by age.
`highest-salary`	Show the employee with the highest salary.
`oldest`	Show the oldest employee.

`payroll <departmentId>`	Compute total payroll for a department.
`bonus <full name>`	Preview the 25% bonus for one employee.
`bonuses`	Preview bonuses for all eligible employees.
`apply-bonus <full name>`	Apply and persist the bonus for one employee.
`apply-bonuses`	Apply and persist bonuses for all eligible employees.
`compare-salary Name A | Name B`	Compare salaries of two employees.
`toggle`	Switch between encrypted and decrypted display mode.
`snapshot [file]`	Export an encrypted snapshot file.
`help`	Print the command list.
`exit` / `quit`	Exit the application.

7.1 Interaction Notes

- The application starts with decryption **OFF**; records are shown exactly as stored in the database.
- Using `toggle` enables decrypted presentation of supported fields.
- `bonus` and `bonuses` are read-only previews.
- `apply-bonus` and `apply-bonuses` persist salary updates back to the encrypted database.
- `compare-salary` expects two names separated by `|`.

8 Configuration and Deployment

8.1 Application Properties

The project is configured through `application.properties`. The main entries are:

Listing 1: Example configuration

```
mongo.uri=mongodb://admin:admin123@localhost:27017/?authSource=admin
mongo.database=company_db
mongo.collection=employees_encrypted
employees.csv.path=Company-Employee.Database/Dataset-Emp-Database.csv
keys.directory=client/index
```

8.2 Running the Project

The project is launched in the following order:

1. Start MongoDB using Docker:

```
cd <project-root>
docker compose up -d
```

2. Move into the `/src` directory.

3. Run the preparation script:

```
./prepare.sh
```

4. Start the client:

```
java -cp ".:lib/*" client/App
```

8.3 Cleanup

To stop the application environment and remove generated artifacts:

```
./clean.sh
docker compose down
```

9 Security Analysis

9.1 Confidentiality

All employee data stored in MongoDB is encrypted. Consequently, compromise of the database server alone is insufficient to recover employee plaintext information.

9.2 Integrity and Authenticity

Each stored record is protected with HMAC-SHA256 and ECDSA-based authenticity checks. This allows the client to detect tampering or unauthorized modification.

9.3 Leakage Discussion

Even though plaintext values are hidden, the server may still observe some leakage:

- Record size and structure.
- Query access patterns.
- Equality leakage from deterministic searchable fields.
- Ordering leakage from OPE-based attributes.
- Frequency information for repeated lookups.

9.4 Leakage Mitigation

Possible mitigations include:

- Reducing the number of deterministic fields.
- Limiting repeated query patterns.
- Padding records or batching requests.
- Using randomized encryption whenever search is not required.
- Keeping sensitive order-dependent values restricted to the minimum necessary.

10 Performance Evaluation

10.1 Bootstrap Latency

Table 3 presents the measured bootstrap execution times for the complete employee dataset under two different conditions: when the required cryptographic keys are already available and when they must be generated during startup.

The results show that the bootstrap process completes in less than half a second in both cases. When the keys are already present, the bootstrap operation requires approximately 229 ms. When key generation is also required, the execution time increases to 454 ms. This difference indicates that key creation is a significant part of the initialization cost.

Even so, the measured latency remains low enough to be practical for repeated executions and demonstrations. Since key generation is usually performed only once, its impact on normal usage is limited.

Dataset Size	Bootstrap Time	Notes
Full dataset (keys given)	229 ms	The time remains fairly constant because the keys were already generated.
Full dataset (keys not given)	454 ms	The operation takes longer because the system must generate or load the required keys.

Table 3: Bootstrap latency measurements

10.2 Encryption Cost

Table 4 summarizes the average execution time observed for each cryptographic mechanism used in the system.

The results show a clear contrast between symmetric encryption and public-key cryptography. OPE is the fastest mechanism, requiring approximately 0.016 ms per operation. AES-GCM and deterministic AES-CTR also have very low latency, remaining below 0.1 ms.

The HMAC and signature generation step adds a moderate overhead of 0.182 ms, which is still small in practice. By far the most expensive primitive is Paillier encryption, which takes approximately 6.21 ms per operation. This is expected because homomorphic encryption relies on much heavier modular arithmetic than symmetric encryption.

Overall, the measurements confirm that the main computational cost comes from the homomorphic encryption component, while the remaining cryptographic mechanisms introduce only minor overhead.

Algorithm	Average Time	Notes
AES-CTR deterministic	0.061 ms	Very fast symmetric encryption used for deterministic protection.
AES-GCM randomized	0.040 ms	Very fast authenticated encryption for confidentiality.
OPE	0.016 ms	Fastest measured primitive; used for ordered queries.
Paillier	6.21 ms	Most expensive primitive; required for additive homomorphic computation.
HMAC / signature generation	0.182 ms	Moderate overhead for integrity and authenticity protection.

Table 4: Encryption and protection overhead

10.3 Query Latency

Table 10.3 presents the latency measurements for the supported client operations.

All operations complete in less than 34 ms, which provides an interactive experience for the user. Exact-match queries such as employee name search and salary comparison are the fastest operations, taking about 5 ms. These operations benefit directly from the searchable encryption structures built during bootstrap.

Department searches and employee ID lookups also remain very fast, staying close to 10 ms. Ordering operations take slightly longer because they require additional processing over the encrypted representations maintained by the client.

The payroll computation operation is also efficient, completing in 16 ms. The highest latency is observed for bonus application, which takes 34 ms because it requires several cryptographic steps, including retrieval, re-encryption, integrity verification, signature regeneration, and database update.

These measurements show that the system can support privacy-preserving database operations with near real-time responsiveness.

Table 5: Query latency measurements

Operation	**Latency**	**Notes**
Search by employee ID	10 ms	Fast lookup using encrypted indexing.
Search by full name	5 ms	Exact-match query with very low latency.
Search by department	9 ms	Efficient encrypted filtering by department.
Salary ordering	17 ms	Requires ordered processing over encrypted salary values.
Age ordering	13 ms	Uses order-preserving encrypted age values.
Highest salary	12 ms	Finds the maximum salary without revealing plaintext values.
Oldest employee	12 ms	Retrieves the employee with the highest encrypted age.
Payroll computation	16 ms	Uses homomorphic addition over encrypted salaries.

Salary comparison	5 ms	Direct comparison supported by encrypted ordering.
Bonus application	34 ms	Includes re-encryption, verification, signing, and persistence.
Snapshot export	9 ms	Writes the encrypted database state to a snapshot file.

10.4 Storage Overhead

Table 6 compares the size of the original plaintext dataset with the encrypted snapshot produced by the system.

The plaintext CSV file occupies approximately 3 KB, while the encrypted snapshot requires approximately 54 KB. This corresponds to an increase of roughly 18 times the original size. The overhead is expected because each record stores multiple encrypted representations of the same data, along with integrity tags, signatures, searchable indexes, and homomorphic ciphertexts.

Although the storage cost is significantly higher than the plaintext version, the absolute size is still small in practical terms. The increase is therefore an acceptable trade-off for supporting confidentiality, integrity, authenticity, searchable encryption, and encrypted computation.

Representation	**Size**	**Difference**	**Ratio**
Plaintext CSV	3 KB	–	1.0×
Snapshot export	54 KB	Encrypted and not human-readable.	18×

Table 6: Storage overhead comparison

10.5 Overall Performance Discussion

The experimental results indicate that the proposed system achieves a practical balance between security and performance. Bootstrap execution completes in under half a second, query latencies remain below 34 ms for all supported operations, and the dominant cryptographic overhead originates from the Paillier homomorphic encryption scheme. While the encrypted dataset requires approximately 18 times more storage than the plaintext representation, this increase is justified by the additional confidentiality, integrity, authenticity, searchable encryption, and homomorphic computation capabilities provided by

the system. Overall, the measured results demonstrate that privacy-preserving database functionality can be achieved with acceptable computational and storage overheads for datasets of this scale.

11 Discussion

The implemented system successfully demonstrates how a practical database can remain encrypted while still supporting a useful set of operations. By combining searchable encryption, order-preserving encryption, homomorphic encryption, integrity verification, and digital signatures, the solution is able to satisfy the functional requirements of the assignment without exposing plaintext employee information to the database server.

One of the main strengths of the system is the breadth of operations supported directly over encrypted data. The implementation supports employee retrieval by identifier and full name, department-based searches, ordering by salary and age, salary comparisons, payroll computation through homomorphic operations, bonus calculations, and encrypted snapshot generation. These operations demonstrate that encrypted databases can provide functionality that goes beyond simple secure storage.

To support these requirements efficiently, different cryptographic mechanisms were applied according to the characteristics of each attribute and operation. Deterministic encryption and searchable indexes were used for exact-match searches, order-preserving encryption was used for ordering and comparison operations, and the Paillier cryptosystem was used whenever arithmetic operations over encrypted salaries were required. Additionally, HMAC-SHA256 and ECDSA signatures were employed to guarantee integrity and authenticity of stored records. This combination of techniques represents a compromise between strong security guarantees and practical performance.

Despite the successful implementation, several trade-offs were necessary. Searchable encryption introduces equality leakage because identical plaintext values generate identical searchable representations. Similarly, order-preserving encryption leaks ordering information, allowing an observer to determine relative ordering between encrypted values. These leakages are inherent to the cryptographic primitives used and are the price paid for supporting efficient search and sorting capabilities. Furthermore, homomorphic encryption introduces significant computational overhead compared to conventional symmetric encryption, as reflected in the performance evaluation results.

The storage evaluation also highlights another important trade-off. The encrypted dataset occupies significantly more space than the original plaintext representation because each record contains multiple encrypted representations, authentication data, and indexing structures. Although this increases storage requirements, the resulting dataset remains manageable for modern storage systems and is justified by the additional functionality provided.

Future work could focus on reducing information leakage while preserving functionality. More advanced searchable encryption schemes could be adopted to hide access patterns and equality relationships. Additional homomorphic capabilities could also be incorporated to support more complex computations over encrypted data. Another possible extension would be the migration from a client-maintained index structure to more sophisticated encrypted indexing mechanisms capable of scaling efficiently to substantially larger datasets. Finally, the system could be extended with distributed deployment, key rotation mechanisms, audit logging, and support for encrypted range queries.

Overall, the implemented solution demonstrates that privacy-preserving databases are practical for real-world scenarios when appropriate cryptographic techniques are carefully selected according to the required operations. The project highlights both the benefits and the limitations of performing computation and search over encrypted information, providing valuable insight into the design of secure outsourced storage systems.

12 Conclusion

This project presented the design and implementation of a privacy-preserving employee database capable of storing information in encrypted form while still supporting a diverse set of query and computation operations. The solution combines deterministic encryption, randomized authenticated encryption, order-preserving encryption, homomorphic encryption, integrity verification, and digital signatures to provide confidentiality, authenticity, and integrity guarantees.

The implemented system successfully satisfies the requirements proposed in the assignment. Employee records can be searched by identifier and name, ordered by salary and age, compared according to salary, grouped by department, and processed through payroll and bonus computations without exposing plaintext information to the database server. Furthermore, all cryptographic keys remain under client control, preventing the server from learning sensitive employee information.

The experimental evaluation demonstrated that the proposed approach is practical in terms of performance. Bootstrap execution completes in under half a second, query latencies remain suitable for interactive usage, and the observed storage overhead is acceptable given the additional cryptographic functionality supported by the system. The results also confirm that homomorphic encryption represents the most computationally expensive component of the design, while searchable encryption and integrity mechanisms introduce only modest overhead.

From a security perspective, the project illustrates the challenges involved in balancing functionality and confidentiality. While some information leakage remains unavoidable due to the use of searchable and order-preserving encryption, the solution effectively protects sensitive employee information from an honest-but-curious database provider and

demonstrates the feasibility of performing meaningful operations over encrypted data.

In conclusion, the project achieved its primary objective of building a secure outsourced employee database capable of supporting practical operations over encrypted information. The implementation serves as a concrete demonstration of how modern cryptographic techniques can be combined to construct secure and functional data management systems.